# NumPy로 만든 다층 신경망

mnist_mlp.py 코드 읽기: 계층 · MultiLayerNet · 미니배치 학습 · 검증과 test

딥러닝개론및연습 · 2026년 2학기

# 이번 시간의 핵심 질문

## 이번 시간의 핵심 질문

- 실습에서 은닉층이 **하나**인 신경망을 계층 클래스로 만들었다 (`mnist_lab.py`)
- 은닉층을 **여러 개** 쌓으려면 새 계층이나 새 역전파 식이 필요할까?
- 여러 신경망 중 어느 것이 좋은지 **무엇으로** 비교해야 할까? test 데이터로 골라도 될까?
- 학습이 끝난 신경망은 **어떤 숫자를** 헷갈릴까?

$\Rightarrow$ `mnist_mlp.py`: **NumPy만으로 만든 다층 신경망 (과제 2의 코드)**

## 학습목표

1. MNIST 데이터를 불러와 펼치기, 정규화, one-hot 인코딩, 훈련/검증 분할을 할 수 있다
2. `Affine`, `Relu`, `Sigmoid`, Softmax-손실 계층의 `forward`와 `backward`를 읽고 shape을 따라갈 수 있다
3. `MultiLayerNet`이 계층을 리스트에 쌓아 순전파와 역전파를 하는 방식을 설명할 수 있다
4. 미니배치 학습 함수 `train`의 epoch, iteration, 갱신을 설명할 수 있다
5. 설정을 **검증 정확도**로 고르고 test는 **한 번만** 평가하는 이유를 설명할 수 있다
6. 혼동행렬을 읽고 `experiments` 리스트를 고쳐 과제 2의 실험을 할 수 있다

# 목차

코드의 전체 구성

데이터 준비

계층 클래스

`MultiLayerNet`: 계층 쌓기

미니배치 학습: `train`

실험: 검증 정확도로 비교

최종 평가: test는 한 번만

정리와 과제 2

1

# 코드의 전체 구성

## mnist_mlp.py의 여섯 부분




- 문제: MNIST $28 \times 28$ 손글씨 숫자 $\rightarrow$ 0–9 분류. **NumPy만** 쓴다 (그림은 matplotlib)
- 계층 클래스(2)는 `mnist_lab.py`와 주석만 빼면 **똑같다**. 새로 생긴 것은 3, 5, 6
- 실행: 파일이 있는 폴더에서 `python mnist_mlp.py` $\rightarrow$ `figures` 폴더에 그림 2개 저장

## `mnist_lab.py`와 무엇이 다른가

| | `mnist_lab.py` **(실습)** | `mnist_mlp.py` **(이번 시간)** |
|---|---|---|
| 신경망 | `TwoLayerNet`: 은닉층 1개 | `MultiLayerNet`: 은닉층 **몇 개든** |
| 계층 클래스 | `Affine`, `Relu`, ⋯ | **같음** |
| 데이터 | 훈련 60000 / test 10000 | 훈련 55000 / **검증 5000** / test 10000 |
| 실험 | 설정 하나 | 여러 설정을 **검증 정확도로 비교** |
| 결과 확인 | 손실 그래프, 예측 이미지 | 정확도 비교 그래프, **혼동행렬** |

- 층을 깊게 쌓는 데 필요한 것은 **새 수식이 아니라 반복문**이다

2

# 데이터 준비

## 1. 데이터 불러오기: load_mnist

```
def load_mnist():
    """Return (x_train, t_train), (x_test, t_test): images (N,28,28), labels (N,)."""
    try:
        from tensorflow.keras.datasets import mnist    # if TensorFlow is installed
        return mnist.load_data()
    except ImportError:
        from sklearn.datasets import fetch_openml      # otherwise download from OpenML
        X, y = fetch_openml("mnist_784", version=1, as_frame=False, return_X_y=True)
        X = X.reshape(-1, 28, 28).astype(np.uint8)
        y = y.astype(np.int64)
        return (X[:60000], y[:60000]), (X[60000:], y[60000:])

(x_all_o, t_all_o), (x_test_o, t_test_o) = load_mnist()
```

- try/except: TensorFlow가 있으면 그것으로, **없으면** scikit-learn으로 내려받는다
- 어느 쪽이든 결과는 같다: 이미지 (N, 28, 28), 픽셀 0–255, 정답 (N,) (클래스 번호)
- 앞 60000장이 훈련용, 뒤 10000장이 test용. 이름 끝의 _o는 **원본**(original)

# 1. 펼치기, 정규화, one-hot

```
x_all  = x_all_o.reshape(x_all_o.shape[0], -1) / 255.0      # (60000, 784)
x_test = x_test_o.reshape(x_test_o.shape[0], -1) / 255.0    # (10000, 784)
t_all  = np.eye(10)[t_all_o]                                 # (60000, 10)
t_test = np.eye(10)[t_test_o]                                # (10000, 10)
```

- `reshape(N, -1)`: 이미지 한 장 $28 \times 28 \to$ 행벡터 하나 ($784$). `-1`은 "나머지를 알아서"
- `/ 255.0`: 픽셀 값을 $0$–$1$로
- `np.eye(10)`: $10 \times 10$ 단위행렬. 그 $k$**번째 행**이 바로 $k$의 one-hot 벡터
- `np.eye(10)[t]`: 행을 정답 번호로 골라 한꺼번에 one-hot

```
labels = np.array([5, 0, 4])
print(np.eye(10)[labels].astype(int))
```

```
[[0 0 0 0 0 1 0 0 0 0]
 [1 0 0 0 0 0 0 0 0 0]
 [0 0 0 0 1 0 0 0 0 0]]
```

# 1. 훈련 / 검증 분할

```
perm = np.random.permutation(x_all.shape[0])
val_idx, train_idx = perm[:5000], perm[5000:]
x_train, t_train = x_all[train_idx], t_all[train_idx]       # (55000, 784)
x_val,   t_val   = x_all[val_idx],   t_all[val_idx]         # (5000, 784)
```

```
train: (55000, 784)  validation: (5000, 784)  test: (10000, 784)
```

| 데이터 | 개수 | 용도 |
|---|---|---|
| 훈련 (train) | 55000 | 매개변수 $W$, $B$를 학습 |
| 검증 (validation) | 5000 | 여러 **설정**(층 수, 넓이, 학습률, …) 중 하나를 고르기 |
| 테스트 (test) | 10000 | 고른 신경망의 최종 성능. **마지막에 한 번만** |

- `permutation`: $0, \ldots, 59999$를 무작위로 섞은 배열. 앞 $5000$개 번호를 검증용으로 뗀다

## 왜 검증 데이터가 따로 필요한가

- 설정을 고를 때 test 정확도를 보면 → **test 데이터에 맞춰** 설정을 고른 셈이 된다
- 그러면 test 정확도는 "처음 보는 데이터"에 대한 성능이 아니게 되고, 실제보다 **좋게** 나온다

**세 데이터의 역할**

1. **훈련 데이터**로 매개변수를 학습한다
2. **검증 데이터**로 여러 설정을 비교해 가장 좋은 것을 고른다
3. 고른 신경망을 **test 데이터**로 **단 한 번** 평가해 보고한다

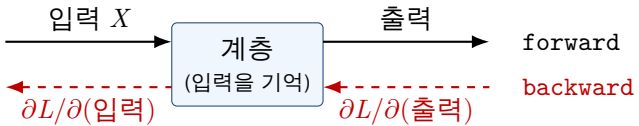
**비유**

검증 데이터는 **모의고사**, test 데이터는 **수능**. 모의고사 점수로 공부 방법을 바꾸는 것은 괜찮지만, 수능 문제를 미리 보고 공부하면 수능 점수는 실력을 나타내지 못한다.

# 3

# 계층 클래스

## 계층의 공통 규칙




- `forward`(입력): 출력을 계산하고, 역전파에 필요한 값을 `self`에 **저장**한다
- `backward`(출력에 대한 기울기): 저장한 값으로 **입력에 대한 기울기**를 돌려준다 (연쇄법칙)
- 매개변수가 있는 계층(`Affine`)은 `backward`에서 `self.dW`, `self.dB`도 저장한다
- 그래서 계층을 **몇 개를 이어 붙이든** 앞에서부터 `forward`, 뒤에서부터 `backward`만 부르면 된다

기호: 미니배치 크기 $h$, 입력 $X \in \mathbb{R}^{h \times l}$, 가중치 $W \in \mathbb{R}^{l \times m}$, 점수 $U$, 정답(one-hot) $T$, softmax 출력 $Y$

## 2. Affine 계층: $A = XW + B$

```python
class Affine:
    def __init__(self, W, B):
        self.W = W          # (l, m)
        self.B = B          # (1, m)
        self.X = None
        self.dW = None
        self.dB = None

    def forward(self, X):
        self.X = X
        return X @ self.W + self.B

    def backward(self, dA):
        self.dW = self.X.T @ dA
        self.dB = np.sum(dA, axis=0, keepdims=True)
        return dA @ self.W.T
```

$$\frac{\partial L}{\partial W} = X^{\mathsf{T}} \frac{\partial L}{\partial A}, \quad \frac{\partial L}{\partial B} = \sum_{\text{행}} \frac{\partial L}{\partial A}$$

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial A}\, W^{\mathsf{T}}$$

| 식 | shape |
|---|---|
| `X @ W + B` | $(h, l)(l, m) + (1, m) \to (h, m)$ |
| `X.T @ dA` | $(l, h)(h, m) \to (l, m)$ |
| `sum(dA,axis=0)` | $(h, m) \to (1, m)$ |
| `dA @ W.T` | $(h, m)(m, l) \to (h, l)$ |

- 기울기의 shape은 **그 변수의 shape과 같다**: `dW`는 `W`와, `dX`는 `X`와
- `+ self.B`: $(1, m)$이 $h$개의 행에 **브로드캐스팅** $\to$ 역전파에서는 행끼리 **더한다**

# 2. 활성화 계층: Relu, Sigmoid

```
class Relu:
    def __init__(self):
        self.mask = None

    def forward(self, A):
        self.mask = (A > 0)
        return np.where(self.mask, A, 0.0)

    def backward(self, dZ):
        return np.where(self.mask, dZ, 0.0)
```

- mask: 입력이 양수였던 자리 (True/False)
- 역전파: 양수였던 자리만 기울기를 **그대로** 통과, 나머지는 $0$

```
class Sigmoid:
    def __init__(self):
        self.Z = None

    def forward(self, A):
        self.Z = 1.0 / (1.0 + np.exp(-A))
        return self.Z

    def backward(self, dZ):
        return dZ * self.Z * (1.0 - self.Z)
```

- $\sigma'(a) = \sigma(a)\big(1 - \sigma(a)\big)$ 이므로 **출력** $Z$만 저장하면 된다
- 모두 **원소별** 연산: shape이 바뀌지 않는다

# 2. softmax: 최댓값을 빼는 이유

```
def softmax(U):
    U = U - np.max(U, axis=1, keepdims=True)
    expU = np.exp(U)
    return expU / np.sum(expU, axis=1, keepdims=
     True)
```

```
U = np.array([[1000.0, 1001.0, 1002.0]])
naive = np.exp(U) / np.sum(np.exp(U), axis=1,
     keepdims=True)
print("naive softmax:", naive)
print("softmax      :", np.round(softmax(U), 4))
```

```
naive softmax: [[nan nan nan]]
softmax      : [[0.09   0.2447 0.6652]]
```

- $e^{1000}$ 은 컴퓨터가 나타낼 수 없다 (`inf`)
  $\rightarrow \infty/\infty =$ `nan`
- 모든 점수에서 같은 수 $c$를 빼도 softmax는 같다:

$$\frac{e^{u_k - c}}{\sum_j e^{u_j - c}} = \frac{e^{u_k}}{\sum_j e^{u_j}}$$

- 행마다 최댓값을 빼면 가장 큰 지수가 $e^0 = 1 \rightarrow$ 넘침이 없다
- `axis=1`: 데이터 하나(**행**)마다

## 2. 출력층: SoftmaxWithCrossEntropy

```
class SoftmaxWithCrossEntropy:
    def __init__(self):
        self.Y = None
        self.T = None

    def forward(self, U, T):
        self.T = T
        self.Y = softmax(U)
        h = U.shape[0]
        return -np.sum(T * np.log(self.Y + 1e-12)) / h

    def backward(self):
        h = self.T.shape[0]
        return (self.Y - self.T) / h
```

$$L = -\frac{1}{h}\sum_{n=1}^{h}\sum_{k=1}^{10} t_{nk}\log y_{nk}$$

$$\frac{\partial L}{\partial U} = \frac{1}{h}(Y - T)$$

- 1e-12: $\log 0 = -\infty$를 피한다
- 손실은 미니배치의 **평균**. 그래서 $h$로 나눈다
- 역전파의 출발점이 **예측 − 정답**

$h$로 나누는 곳은 **여기 한 곳뿐**이다. 앞 계층들은 이 $\frac{1}{h}$이 곱해진 기울기를 받아서 그대로 전달하므로, Affine의 dW, dB도 자동으로 미니배치 평균의 기울기가 된다.

## 2. 비교용: SoftmaxWithSquaredError

```
class SoftmaxWithSquaredError:
    def forward(self, U, T):
        self.T = T
        self.Y = softmax(U)
        h = U.shape[0]
        return 0.5 * np.sum((self.Y - self.T) ** 2) / h

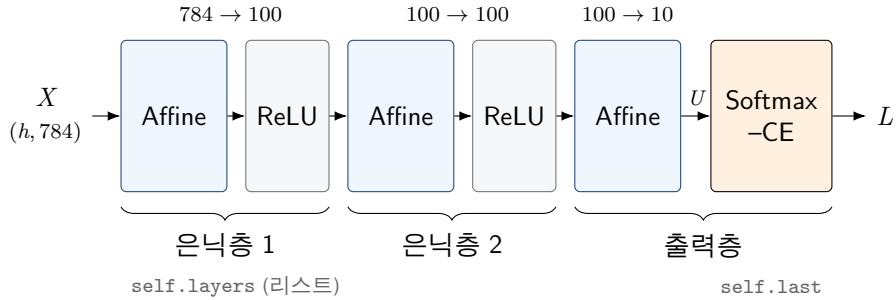
    def backward(self):
        h = self.T.shape[0]
        R = self.Y - self.T
        return self.Y * (R - np.sum(self.Y * R, axis=1, keepdims=True)) / h
```

- softmax 출력에 **제곱 오차**를 쓴 손실 (__init__은 위와 같아서 생략)
- 역전파 식이 $\frac{1}{h}(Y - T)$처럼 간단하지 않다: softmax의 미분이 그대로 남는다
- 과제 2에서 loss="se"로 바꾸면 이 계층을 쓴다. cross entropy와 학습 속도를 비교해 보자

4

# MultiLayerNet: 계층 쌓기

# 구조: Affine과 활성화를 번갈아 쌓는다




`MultiLayerNet(784, [100, 100], 10)`: 은닉층 2개

- 앞 층의 **모든** node가 다음 층의 **모든** node와 연결: **완전 연결 신경망** (다층 퍼셉트론, MLP)
- 은닉층의 **넓이**(node 수)와 **깊이**(층 수)를 `hidden_sizes` 리스트 하나로 정한다
- 마지막 Affine 뒤에는 활성화가 **없다**: 점수 $U$가 바로 Softmax-손실 계층으로

# 3. `__init__`: 층의 크기 목록으로 계층 만들기

```
class MultiLayerNet:
    def __init__(self, input_size, hidden_sizes, output_size,
                 activation="relu", loss="ce", weight_init_std=0.05):
        sizes = [input_size] + list(hidden_sizes) + [output_size]   # e.g. [784, 256, 128, 10]
        self.params = {}
        self.layers = []
        for i in range(len(sizes) - 1):
            n_in, n_out = sizes[i], sizes[i + 1]
            W = weight_init_std * np.random.randn(n_in, n_out)   # (input nodes, output nodes)
            B = np.zeros((1, n_out))
            self.params[f"W{i + 1}"] = W
            self.params[f"B{i + 1}"] = B
            self.layers.append(Affine(W, B))
            if i < len(sizes) - 2:                       # no activation after the last Affine
                self.layers.append(Relu() if activation == "relu" else Sigmoid())
        self.last = SoftmaxWithCrossEntropy() if loss == "ce" else SoftmaxWithSquaredError()
```

- `sizes`의 이웃한 두 수 $(n_{\text{in}}, n_{\text{out}})$마다 Affine 하나. 크기가 $k+2$개면 Affine은 $k+1$개
- 가중치: 표준편차 $0.05$의 정규분포 난수, 편향: $0$. `f"W{i + 1}"`: 이름 `"W1"`, `"W2"`, …

# 3. 만들어진 계층과 매개변수 확인

```
net = MultiLayerNet(784, [100, 100], 10)
print([type(layer).__name__ for layer in net.layers
    ])
print(type(net.last).__name__)
for key, p in net.params.items():
    print(f"{key}: {p.shape}")
print(net.num_params())
```

```
['Affine', 'Relu', 'Affine', 'Relu', 'Affine']
SoftmaxWithCrossEntropy
W1: (784, 100)
B1: (1, 100)
W2: (100, 100)
B2: (1, 100)
W3: (100, 10)
B3: (1, 10)
89610
```

매개변수 수:

$$\underbrace{784 \cdot 100 + 100}_{\text{Affine 1}} + \underbrace{100 \cdot 100 + 100}_{\text{Affine 2}} + \underbrace{100 \cdot 10 + 10}_{\text{Affine 3}} = 89610$$

- `num_params()`: 모든 배열의 원소 수 (`p.size`)의 합
- 은닉층 하나 `[100]`이면 79510개
- ReLU에는 매개변수가 **없다**

# 3. 순전파: predict, loss, accuracy

```
def predict(self, X):                          # scores
    U
    for layer in self.layers:
        X = layer.forward(X)
    return X

def loss(self, X, T):
    return self.last.forward(self.predict(X), T)

def accuracy(self, X, T):
    U = self.predict(X)
    return np.mean(np.argmax(U, axis=1)
                   == np.argmax(T, axis=1))
```

**미니배치 3개가 지나가는 shape**

```
input (3, 784)
Affine   -> (3, 100)
Relu     -> (3, 100)
Affine   -> (3, 100)
Relu     -> (3, 100)
Affine   -> (3, 10)
```

- `predict`: 리스트의 계층을 **앞에서부터** 차례로 통과. 변수 `X`를 계속 덮어쓴다
- `accuracy`: 점수가 가장 큰 클래스 = 정답 클래스인 비율. softmax를 하지 않아도 `argmax`는 같다
- `loss`를 부르면 모든 계층이 역전파에 필요한 값을 **저장**해 둔다 → 바로 다음에 `backward`

## 3. 역전파: backward

```
def backward(self):                          # call right after loss()
    d = self.last.backward()
    for layer in reversed(self.layers):      # the same rule, layer after layer
        d = layer.backward(d)
    grads, k = {}, 1
    for layer in self.layers:
        if isinstance(layer, Affine):
            grads[f"W{k}"], grads[f"B{k}"] = layer.dW, layer.dB
            k += 1
    return grads
```

- 출발은 출력층의 $\frac{1}{h}(Y - T)$. reversed로 계층 목록을 **뒤에서부터** 돈다
- 각 계층은 받은 기울기로 자기 입력의 기울기를 만들어 앞 계층에 넘긴다
- Affine 계층의 dW, dB를 모아 params와 **같은 이름**의 딕셔너리로 (isinstance: ReLU는 건너뜀)
- 층이 몇 개이든 **같은 규칙을 반복**할 뿐이다. 깊은 신경망의 역전파에 **새로운 수식은 필요 없다**

# 3. 기울기 검사: 역전파가 맞는지 확인

```
small = MultiLayerNet(5, [4, 3], 3, weight_init_std
    =0.5)
# (biases set to small nonzero values)
Xs = np.random.randn(2, 5)
Ts = np.eye(3)[[0, 2]]
small.loss(Xs, Ts)
grads = small.backward()
# numerical: (L(p + eps) - L(p - eps)) / (2 eps)
```

```
gradient check (max |backprop - numerical|):
W1: 1.05e-11
B1: 5.61e-12
W2: 4.53e-12
B2: 7.32e-13
W3: 1.29e-11
B3: 9.91e-12
```

- 은닉층 2개인 작은 신경망 $(5 \to 4 \to 3 \to 3)$
- 모든 매개변수에서 차이가 $10^{-11}$ 정도 $\to$ 역전파가 **맞다**
- 코드: `mlp_demo.py`

**편향을 0으로 두면?**

B2에서만 차이가 $9 \times 10^{-2}$로 컸다. 첫 은닉층의 ReLU 출력이 한 행에서 모두 $0$이면 둘째 Affine의 출력이 정확히 $0$, 즉 ReLU가 **꺾이는 점**에 놓여 수치 미분이 틀린다.

5

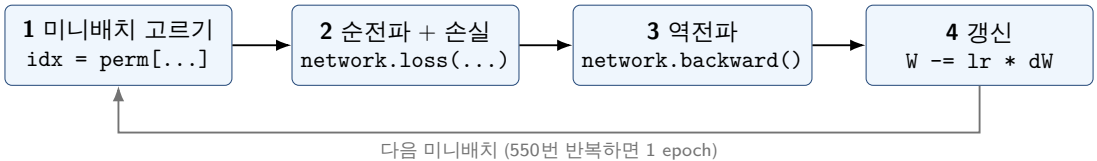
# 미니배치 학습: `train`

# 4. train 함수

```
def train(network, epochs=20, batch_size=100, learning_rate=0.1, name=""):
    N = x_train.shape[0]
    iter_per_epoch = N // batch_size
    hist = {"loss": [], "train_acc": [], "val_acc": []}
    t0 = time.time()
    for epoch in range(epochs):
        perm = np.random.permutation(N)
        for it in range(iter_per_epoch):
            idx = perm[it * batch_size:(it + 1) * batch_size]
            loss = network.loss(x_train[idx], t_train[idx])
            grads = network.backward()
            for key in network.params:
                network.params[key] -= learning_rate * grads[key]
            hist["loss"].append(loss)
        hist["train_acc"].append(network.accuracy(x_train, t_train))
        hist["val_acc"].append(network.accuracy(x_val, t_val))
        print(...)
    return hist
```

- `N // batch_size`: $55000//100 = 550$ iteration이 1 epoch. epoch마다 **새로** 섞고 $100$개씩 잘라 쓴다

## 한 iteration에서 일어나는 일




| 용어 | 이 코드에서 |
|---|---|
| 미니배치 크기 | `batch_size` $= 100$ |
| iteration | 매개변수를 한 번 갱신 (미니배치 하나) |
| epoch | 훈련 데이터 $55000$개를 **모두 한 번씩** 사용 $= 550$ iteration |

- 정확도는 epoch이 끝날 때마다 훈련 데이터 **전체**와 검증 데이터 전체에서 계산한다

# 갱신은 반드시 -=로

```
network.params[key] -= learning_rate * grads[key]
```

- `self.params["W1"]`과 첫 Affine 계층의 `self.W`는 **같은 배열**이다 (`__init__`에서 같은 `W`를 넣었다)
- `-=`는 그 배열의 숫자를 **제자리에서** 바꾼다 → 계층도 바뀐 가중치를 본다

```
print(net.params["W1"] is net.layers[0].W)
net.params["W1"] -= 0.1                # in-place
# -> the layer sees the change?
net.params["W1"] = net.params["W1"] - 0.1
# -> the layer sees the change?
```

```
params['W1'] is layers[0].W : True
in-place  -= : layer sees it -> True
new array  = : layer sees it -> False
```

**=로 쓰면?**

`W = W - lr * dW`는 **새 배열**을 만들어 딕셔너리에만 넣는다. 계층은 옛 가중치를 계속 쓰므로 오류 없이 실행되지만 **학습이 전혀 되지 않는다**.

6

# 실험: 검증 정확도로 비교

## 5. experiments 리스트

```
experiments = [
    ("A: [100]",       [100],      "relu", "ce", 0.1, 100),   # the network of the lab (Step 5)
    ("B: [100, 100]", [100, 100], "relu", "ce", 0.1, 100),   # one more hidden layer
]
epochs = 10

results = {}
for name, hidden, act, loss_type, lr, bs in experiments:
    np.random.seed(1)
    net = MultiLayerNet(784, hidden, 10, activation=act, loss=loss_type)
    print(f"\n{name}: {net.num_params()} parameters")
    hist = train(net, epochs=epochs, batch_size=bs, learning_rate=lr, name=name.split(":")[0])
    results[name] = (net, hist)
```

- 한 줄이 실험 하나: (이름, 은닉층 넓이들, 활성화, 손실, 학습률, 미니배치 크기)
- for 문이 튜플을 여섯 변수로 **풀어서** 받는다
- np.random.seed(1): 실험마다 같은 난수에서 출발 → 차이가 **설정 때문**임을 보장
- results: 이름 → (학습된 신경망, 기록). 가장 좋은 것을 고를 때 쓴다

## 5. 실행 결과

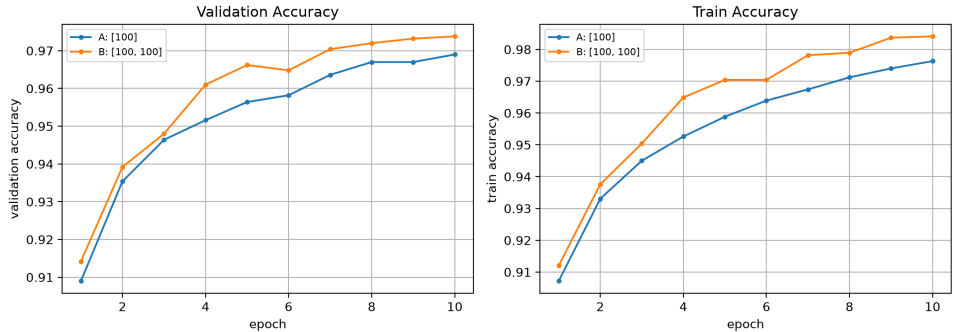
```
A: [100]: 79510 parameters
[A] epoch  1: loss=0.3415, train acc=0.9072, val acc=0.9090 (1s)
  ...
[A] epoch  9: loss=0.1002, train acc=0.9740, val acc=0.9670 (7s)
[A] epoch 10: loss=0.1476, train acc=0.9763, val acc=0.9690 (8s)

B: [100, 100]: 89610 parameters
[B] epoch  1: loss=0.3214, train acc=0.9120, val acc=0.9142 (1s)
  ...
[B] epoch  9: loss=0.0504, train acc=0.9837, val acc=0.9732 (10s)
[B] epoch 10: loss=0.0509, train acc=0.9841, val acc=0.9738 (11s)

===== summary (validation accuracy after the last epoch) =====
A: [100]                      params=  79510  train=0.9763  val=0.9690
B: [100, 100]                 params=  89610  train=0.9841  val=0.9738
```

- 은닉층을 하나 더 쌓은 B가 검증 정확도 $97.38\%$로 A($96.90\%$)보다 높다. 매개변수는 $13\%$ 많다
- 출력된 `loss`는 그 epoch의 **마지막 미니배치 하나**의 손실이라 오르락내리락한다
- NumPy만으로 CPU에서 실행: 10 epoch에 A는 8초, B는 11초

# 5. 정확도 비교 그래프: `mlp_compare.png`




왼쪽: epoch마다의 검증 정확도, 오른쪽: 훈련 정확도

- 두 신경망 모두 10 epoch까지 정확도가 **아직 오르고 있다** → epoch를 늘려 볼 만하다
- 훈련 정확도와 검증 정확도의 차이: A는 $0.73\%$p, B는 $1.03\%$p. 신경망이 클수록 차이가 커지는지 지켜보자 (과적합)

7

# 최종 평가: test는 한 번만

## 6. 검증 정확도로 고르고, test는 한 번

```
best_name = max(results, key=lambda k: results[k][1]["val_acc"][-1])
best_net = results[best_name][0]
test_acc = best_net.accuracy(x_test, t_test)
print(f"\nbest on validation: {best_name}")
print(f"test accuracy of the chosen network: {test_acc:.4f}")
```

```
best on validation: B: [100, 100]
test accuracy of the chosen network: 0.9733
```

- `max(results, key=...)`: 딕셔너리의 키(이름) 중에서 `key` 함수의 값이 가장 큰 것
  - `results[k][1]`: 기록 `hist`, `["val_acc"][-1]`: 마지막 epoch의 검증 정확도
- test 정확도는 **고른 신경망 하나에 대해서만** 계산한다: $97.33\%$
- 검증 정확도 $97.38\%$와 비슷하다 $\rightarrow$ 검증 데이터로 고른 것이 처음 보는 데이터에도 통했다

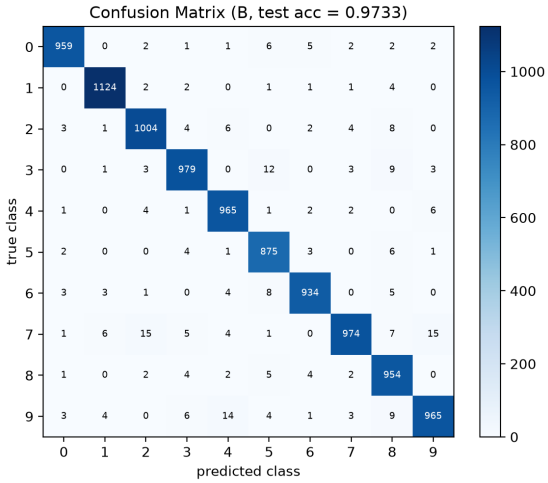
## 6. 혼동행렬 만들기

```
pred = np.argmax(best_net.predict(x_test), axis=1)
C = np.zeros((10, 10), dtype=int)
for t, p in zip(t_test_o, pred):
    C[t, p] += 1
...
print("per-class test accuracy:", np.round(np.diag(C) / C.sum(axis=1), 4))
```

```
per-class test accuracy: [0.9786 0.9903 0.9729 0.9693 0.9827 0.9809 0.9749 0.9475 0.9795 0.9564]
```

- **혼동행렬** $C$: $C_{ij}$ = "정답이 $i$인데 $j$로 예측한 test 이미지의 수"
- `zip(t_test_o, pred)`: (정답, 예측)의 쌍을 하나씩. 여기서는 one-hot이 아닌 **원래 번호**를 쓴다
- 대각선 `np.diag(C)`: 맞힌 수. 행의 합 `C.sum(axis=1)`: 그 숫자의 test 이미지 수
- 클래스별 정확도가 가장 낮은 숫자: **7** ($94.75\%$), 그다음 **9** ($95.64\%$)

# 6. 혼동행렬: `mlp_confusion.png`




- 대각선이 진하다: 대부분 맞힌다
- 대각선 밖에서 큰 칸 (정답 $\to$ 예측):
  - $7 \to 2$: 15장, $7 \to 9$: 15장
  - $9 \to 4$: 14장
  - $3 \to 5$: 12장
- 모양이 비슷한 숫자끼리 헷갈린다
- 혼동행렬은 대칭이 아니다: $7 \to 2$는 15장이지만 $2 \to 7$은 4장

틀린 이미지를 직접 그려 보면 사람도 헷갈릴 만한 것이 많을까?

8

# 정리와 과제 2

# 정리

## 이번 시간의 핵심

- **데이터**: `reshape(N, -1)`로 펼치고 `/255`로 정규화, `np.eye(10)[t]`로 one-hot. 훈련 55000 / 검증 5000 / test 10000
- **계층**: `forward`는 값을 저장하고 `backward`는 저장한 값으로 입력의 기울기를 돌려준다. 기울기의 shape $=$ 변수의 shape
- **출력층**: softmax는 최댓값을 빼고 계산. cross entropy와 함께 쓰면 역전파의 출발이 $\frac{1}{h}(Y - T)$
- **`MultiLayerNet`**: `sizes` 목록으로 Affine–활성화를 리스트에 쌓는다. 순전파는 앞에서부터, 역전파는 `reversed`
- **학습**: epoch마다 섞고 100개씩. 갱신은 `-=` (계층과 같은 배열을 제자리에서)
- **평가**: 설정은 **검증 정확도**로 고르고, test는 고른 신경망에 **한 번만**. 혼동행렬로 어디서 틀리는지 본다
- 결과: `[100, 100]`이 검증 $97.38\%$로 선택, test $97.33\%$

# 과제 2: MNIST 정확도 98% 도전

`experiments` 리스트에 한 줄씩 추가하며 실험하자. 한 번에 **한 가지만** 바꾼다.

| 바꿀 것 | 어디를 | 예 |
|---|---|---|
| 깊이 (은닉층 수) | 둘째 항목 | `[100, 100, 100]` |
| 넓이 (node 수) | 둘째 항목 | `[300]`, `[256, 128]` |
| 활성화 함수 | 셋째 항목 | `"sigmoid"` |
| 손실 함수 | 넷째 항목 | `"se"` (제곱 오차) |
| 학습률 | 다섯째 항목 | `0.05`, `0.2` |
| 미니배치 크기 | 여섯째 항목 | 50, 200 |
| epoch 수 | `epochs = 10` | 20, 30 |

- 시도한 설정과 결과(train/검증 정확도, 그래프)를 **표로** 정리하고, test 정확도는 **최종 선택한 신경망만** 보고
- 어떤 요인의 영향이 가장 컸는지, 과적합의 징후(train과 검증의 차이)가 있었는지 설명

# 실습 문제

1. `MultiLayerNet(784, [256, 128], 10)`의 매개변수 수를 손으로 계산하고 `num_params()`로 확인하기
2. `train`이 epoch마다 **평균 손실**을 출력하도록 고치기 (힌트: `hist["loss"]`의 마지막 `iter_per_epoch`개)
3. 갱신 줄을 `network.params[key] = network.params[key] - learning_rate * grads[key]`로 바꾸면 정확도가 어떻게 되는가? 이유는?
4. 은닉층 넓이는 그대로 두고 깊이만 $1, 2, 3, 4$로 늘려 검증 정확도와 실행 시간을 비교하기
5. 매개변수 수가 비슷한 **넓은** 신경망(`[113]` 정도)과 **깊은** 신경망(`[100, 100]`)을 비교하기
6. 혼동행렬에서 $7 \to 2$로 틀린 test 이미지를 찾아 그리기 (힌트: `np.where((t_test_o == 7) & (pred == 2))`)
7. `weight_init_std`를 $0.5$와 $0.001$로 바꾸고 `[100, 100, 100, 100]`을 학습시키면? (가중치 초기화가 왜 중요한가)